Guia de estudos de React para quem vem de Vue

React, explicado através de um projecto real para quem já conhece Vue

Este guia usa o CVRápido como laboratório. A aplicação está escrita em React 18.3, TypeScript e JSX. Em vez de aprender React como uma lista de APIs, siga um dado desde o formulário até ao estado partilhado, ao preview e à persistência.

Como usar este guia

1. Abra o projecto e execute `npm install` e `npm run dev`.
2. Leia uma secção de cada vez e acompanhe os ficheiros indicados.
3. Faça os exercícios numa branch própria ou guarde as alterações antes de começar; o projecto é uma aplicação funcional.
4. Depois de cada exercício, execute `npm run build` e experimente o fluxo no navegador.

Não tente traduzir cada API de Vue para uma API com o mesmo nome em React. Primeiro identifique quem possui o estado, o que causa uma nova renderização e se o código produz interface ou executa um efeito externo.

Mapa de orientação no projecto

Conceito a estudar	Onde observar
Componente raiz e composição	`src/app/App.tsx`
Estado partilhado e contexto	`src/app/context/ResumeContext.tsx`
Hook de conveniência do domínio	`src/app/hooks/useResumeData.ts`
Navegação por etapas e efeitos	`src/app/components/ResumeWizard.tsx`
Campo controlado e evento	`src/app/components/forms/PersonalDataForm.tsx`
Colecção, edição e chaves	`src/app/components/forms/ExperienceForm.tsx`
Renderização derivada dos dados	`src/app/components/ResumePreview.tsx`
Formas dos dados em TypeScript	`src/app/types/resume.ts`
Persistência e integração com o navegador	`src/app/utils/resumeStorage.ts`

O modelo mental essencial

Em Vue, é comum pensar num componente como uma instância que acompanha valores reactivos. Em React, pense numa função de renderização:

1. React chama o componente com as suas `props`.
2. O componente lê o estado e calcula o JSX a apresentar.
3. Quando o estado muda, React volta a chamar o componente.
4. React compara a nova árvore e actualiza o DOM necessário.

O JSX descreve a interface para o estado actual; não é uma sequência de instruções para alterar o DOM. Não edite o DOM manualmente para reflectir o estado da aplicação.

Correspondências Vue → React

Vue	React neste projecto	Diferença importante
`<script setup>`	Função de componente (`function PersonalDataForm()`)	Cada renderização volta a executar a função.
`defineProps()`	Propriedades recebidas como argumento	São entradas; não devem ser alteradas pelo componente filho.
`emit('change', value)`	Callback recebido por `props`, por exemplo `onBack`	A função é passada do componente pai para o filho.
`ref()` / `reactive()`	`useState()`	O setter substitui o estado; objectos e arrays devem ser actualizados imutavelmente.
`computed()`	Expressão derivada durante a renderização	Use `useMemo()` apenas quando houver uma razão de desempenho ou identidade estável.
`watch()` / `watchEffect()`	Muitas vezes nada; para efeitos externos, `useEffect()`	`useEffect` não é substituto directo de `computed`; derivações simples devem ser calculadas no render.
`onMounted()` / `onUnmounted()`	`useEffect()` com dependências e função de limpeza	O efeito pode voltar a correr quando as dependências mudam.
`provide` / `inject`	`Context.Provider` / `useContext()`	O valor do contexto é partilhado pelos descendentes do Provider.
Composable `useX()`	Custom Hook `useX()`	Deve chamar Hooks sempre no topo de componentes ou de outros Hooks.
`v-if`	Condição JavaScript no JSX	Por exemplo, `condicao ? <A /> : <B />`.
`v-for`	`.map()`	Cada elemento da lista precisa de uma `key` estável.
`v-model`	Valor controlado + `onChange`	O estado é a fonte de verdade e recebe cada alteração.
`<slot>`	`props.children` ou uma prop de renderização	A composição explícita é a forma habitual.

Template e JSX

Vue:

```
<input v-model="personalData.fullName">
<p v-if="personalData.email">{{ personalData.email }}</p>
<li v-for="experience in experiences" :key="experience.id">
  {{ experience.title }}
</li>
```

React:

```
<input
  value={personalData.fullName}
  onChange={(event) => updatePersonalData({ fullName: event.target.value })}
/>
{personalData.email && <p>{personalData.email}</p>}
{experiences.map((experience) => (
  <li key={experience.id}>{experience.title}</li>
))}
```

Em JSX, as expressões JavaScript entram entre `{}`. Usa-se `className` em vez de `class`, e os eventos são propriedades camelCase, como `onChange` e `onClick`.

1. Componentes, props e composição

Comece por `App.tsx`. App mantém a escolha do ecrã activo e compõe `LandingPage`, `ResumeWizard` e `ResumeAnalysisForm`. Não há um router neste fluxo: uma união de estados e renderização condicional escolhem a vista.

`ResumeWizard` recebe `initialMode` e `onBack`. Isto é semelhante a definir props e emitir um evento para o pai, mas o evento React é simplesmente uma função:

```
interface ResumeWizardProps {
  onBack: () => void;
  initialMode?: ResumeStartMode;
}

export function ResumeWizard({ onBack, initialMode = 'empty' }: ResumeWizardProps) {
  // ...
}
```

Prática: em `App.tsx`, siga `onStart` da página inicial até à mudança do serviço activo e à passagem de `initialMode` para o wizard. Identifique qual componente possui cada estado.

2. Estado local e renderizações

`ResumeWizard` usa `useState` para o passo actual, a personalização e os erros de validação. O par devolvido por `useState` é o valor actual e uma função que agenda a sua actualização:

```
const [currentStep, setCurrentStep] = useState(0);
setCurrentStep(currentStep + 1);
```

Não altere um objecto ou array existente em lugar. Crie um novo valor; React usa essa nova referência para detectar actualizações. Quando o próximo estado depende do anterior, prefira a forma funcional:

```
setCurrentStep((step) => step + 1);
```

É especialmente importante quando actualizações podem ser agrupadas ou quando o valor anterior pode estar desactualizado.

Prática: encontre `currentStep`, `showCustomization` e `validationErrors` em `ResumeWizard.tsx`. Para cada um, explique qual acção do utilizador o altera e que parte do JSX muda em resposta.

3. Campos controlados (equivalente prático de `v-model`)

Em `PersonalDataForm.tsx`, o `value` de cada campo vem de `resumeData`, e `onChange` chama `updatePersonalData`. O input não mantém uma cópia independente do nome: o estado partilhado é a fonte de verdade.

Em Vue, `v-model` combina leitura e escrita do valor. Em React, escrevemos as duas direcções:

```
<Input
  value={personalData.fullName}
  onChange={(event) =>
    updatePersonalData({ fullName: event.target.value })
  }
/>
```

Para checkboxes, o valor controlado é `checked`; para um `<select>`, é normalmente `value`. O evento é explícito.

Prática: siga o campo de email desde `PersonalDataForm` até `updatePersonalData` em `ResumeContext`. Confirme onde o preview recebe esse mesmo dado.

4. Estado partilhado: Context e custom Hooks

`ResumeContext.tsx` define o estado do CV e as operações que o alteram. `ResumeProvider` envolve a aplicação em `App.tsx`; descendentes acedem a esse valor com `useResume()`. O Hook lança um erro claro quando usado fora do Provider.

O projecto mantém o estado central porque o formulário, a personalização e o preview precisam dos mesmos dados. Em vez de importar o contexto em cada formulário, `useResumeData()` expõe as operações de domínio utilizadas pelos formulários.

Isto aproxima-se de `provide/inject` mais um composable, mas sem proxy reactivo: é o contexto que distribui o estado e os setters que produzem novas versões dele. Context é conveniente para esta escala; não é obrigatório para todo estado React.

Prática: desenhe o fluxo `ResumeProvider → PersonalDataForm → useResumeData → useResume → Context`. Depois encontre onde `ResumePreview` recebe `resumeData` e `settings`.

5. Actualizações imutáveis de objectos e arrays

Em `ResumeContext`, uma actualização parcial dos dados pessoais conserva os outros campos através do spread:

```
setResumeData((previous) => ({
  ...previous,
  personalData: { ...previous.personalData, ...data },
}));
```

Uma colecção é actualizada por uma nova lista. `addExperience` acrescenta com spread; `updateExperience` usa `map`; `deleteExperience` usa `filter`. Nenhuma destas operações altera directamente `previous.experiences`.

Em Vue, mutações em estado profundamente reactivo podem ser observadas pelo sistema. Em React, não faça `resumeData.personalData.fullName = value` nem `experiences.push(item)`; chame a operação/setter com novos objectos/arrays. Quando depende do estado anterior, a forma `setState(previous => next)` evita usar um snapshot antigo.

Prática: compare adicionar, editar e apagar uma experiência em `ExperienceForm.tsx` e `ResumeContext.tsx`.

6. Listas, condições e identidade

React usa JavaScript normal para decidir o que renderizar:

1. `condicao && <Componente />` para mostrar ou não um bloco;
1. `condicao ? <A /> : <B />` para escolher entre alternativas;
1. `items.map(...)` para produzir elementos repetidos.

No formulário de experiência, cada cartão recebe `key={exp.id}`. A chave deve ser estável e única entre irmãos; não use o índice se a lista puder ser reordenada, editada ou removida. Ela permite ao React preservar a identidade do item correcto.

Prática: localize a lista `experiences.map` e descreva o que poderia acontecer com o foco/campo de edição se os itens não tivessem uma chave estável.

7. Renderização derivada e `computed`

`ResumePreview.tsx` recebe `data` e `settings` como props e escolhe layout e secções a apresentar. O preview é calculado a partir das props: não precisa de copiar o currículo para estado local nem de sincronizar essa cópia com um watcher.

Este é o primeiro equivalente a estudar para `computed`: uma expressão ou variável normal durante a renderização:

```
const safeColor = normalizeColor(primaryColor);
const isModern = settings.template === 'modern' && !settings.atsMode;
```

`useMemo` pode evitar uma operação cara ou manter identidade referencial quando isso é importante; não o use como tradução automática de cada `computed`.

8. `useEffect`, ciclo de vida e efeitos externos

`useEffect` serve para sincronizar React com algo externo à renderização: armazenamento do navegador, subscrições, timers ou APIs. Em `ResumeContext.tsx`, o efeito persiste o estado quando os dados ou as definições mudam:

```
useEffect(() => {
  saveStoredResume(resumeData, settings);
}, [resumeData, settings]);
```

O array de dependências declara os valores que o efeito utiliza e que, quando mudam, o fazem voltar a correr. Uma função de limpeza devolvida pelo efeito serve para libertar subscrições/timers, quando existirem.

Não use um efeito para calcular algo que pode ser obtido directamente durante a renderização. Isso cria estado duplicado, atrasos e risco de inconsistência. A decisão `isDirectMode` em `ResumeWizard` é derivada de `initialMode`; não precisa de ser guardada em estado.

`useEffect` não corresponde exactamente a `watch` nem a `onMounted`: o efeito corre depois de React confirmar a renderização e volta a sincronizar quando as dependências mudam. Se a aplicação for envolvida em `StrictMode`, React pode repetir o ciclo de efeitos em desenvolvimento para revelar efeitos não seguros; por isso, efeitos devem tolerar limpeza e inicialização repetidas. Actualmente, `src/main.tsx` não envolve a aplicação em `StrictMode`.

Prática: compare o efeito de persistência do Provider com o efeito de inicialização de modo em `ResumeWizard`. Para cada um, indique qual sistema externo ou estado partilhado está a sincronizar.

9. Hooks, callbacks e referências

Regras fundamentais dos Hooks:

1. Chame Hooks no topo de um componente React ou de outro Hook.
2. Não os chame dentro de `if`, ciclos ou funções chamadas em momentos arbitrários.

3 Mantenha as dependências de `useEffect`, `useCallback` e `useMemo` sincronizadas com os valores lidos pelo callback.

`useCallback` em `ResumeContext` conserva a identidade de operações enquanto as dependências não mudam. `useMemo` conserva o objecto `value` do Provider entre renderizações com as mesmas dependências. São optimizações e ferramentas de identidade; não tornam uma função “mais correcta” por si só.

`useRef` guarda um valor mutável que não precisa de provocar renderização, ou fornece uma referência a um elemento DOM. `ResumePreview` está envolvido em `forwardRef` e pode expor o elemento raiz, embora o wizard não passe actualmente uma ref para ele. Use estado quando a mudança precisa de actualizar a interface; use ref para acesso imperativo ou um valor que não participa no desenho.

10. TypeScript e JSX

Os tipos de domínio estão em `src/app/types/resume.ts`. Por exemplo, `ResumeData` descreve o currículo e `ResumeSettings` descreve as preferências. As props dos componentes também podem ser tipadas com interfaces ou tipos. `type` imports trazem apenas informação de compilação:

```
import type { ResumeData, ResumeSettings } from '@/app/types/resume';
```

O alias `@/` aponta para `src/`, configurado no Vite e no `tsconfig.json`. TypeScript verifica os tipos, mas não valida automaticamente dados JSON em runtime; para conteúdo importado, observe a validação em `resumeStorage.ts`.

11. Plano prático em sete sessões

Sessão 1 — Renderização e componentes

1 Leia `App.tsx` e `LandingPage.tsx`.

1 Desenhe a árvore de componentes até `ResumeWizard`.

1 Exercício: explique a diferença entre uma prop e o estado local de `App`.

Sessão 2 — Estado local e eventos

1 Estude `useState`, `currentStep` e os manipuladores do wizard.

1 Exercício: acompanhe um clique em **Próximo** desde o evento até à etapa apresentada.

Sessão 3 — Formulários

1 Estude `PersonalDataForm.tsx` e `SummaryForm.tsx`.

1 Exercício: siga uma alteração de nome até aparecer no preview.

Sessão 4 — Estado partilhado

1 Leia `ResumeContext.tsx` e `useResumeData.ts`.

1 Exercício: trace a operação de adicionar uma experiência e a respectiva apresentação no preview.

Sessão 5 — Arrays e componentes de lista

1 Leia `ExperienceForm.tsx`: `map`, `filter`, chaves e estado do formulário.

1 Exercício: explique como o mesmo formulário alterna entre adicionar e editar.

Sessão 6 — Efeitos e persistência

1 Leia a inicialização do Provider e `resumeStorage.ts`.

1. Exercício: altere um campo, recarregue a página e explique a sequência que conserva os dados.

Sessão 7 — Composição e renderização

1. Leia `ResumePreview.tsx` e os layouts.
1. Exercício: trace como mudar `settings.layout` selecciona uma disposição, sem introduzir estado duplicado.

Exercícios finais

Faça-os um de cada vez; depois de cada mudança, confirme preview, reload e build:

1. Adicione um campo simples ao tipo `PersonalData`, ao formulário, à operação `updatePersonalData` (se necessário), ao preview e ao PDF.
2. Adicione um botão para duplicar uma experiência, gerando um novo `id` em vez de reutilizar a chave da experiência original.
3. Calcule e apresente um número derivado de competências sem o guardar em estado.
4. Extraia uma pequena secção reutilizável que receba dados por props e confirme que não altera os dados recebidos.

Referências úteis

1. React: Descrever a UI
1. React: Adicionar interactividade
1. React: Gerir estado
1. React: Escape hatches — Effects e refs
1. React com TypeScript

O código deste projecto é a referência final para o comportamento da aplicação; os exercícios são propostas de estudo e não exigem alterações permanentes ao produto.